“Linux Process Monitoring and Control System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

P. RUHITHA	**2520080053**
M. ANANYA	**2520080069**
CH. MADHULIKA	**2520090056**

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled **"Design and Development of an Integrated Tourism Platform for Flights, Trains, Hotels, and Hidden Destinations"** is a record of Bonafide work of **P.RUHITHA – 2520080053 , M.ANANYA - 2520080069 CH. MADHULIKA - 2520090056,** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

P. RUHITHA	**2520080053**
M. ANANYA	**2520080069**
CH. MADHULIKA	**2520090056**

K L (Deemed to be) University

DEPARTMENT OF CS-IT & AI&DS



CERTIFICATE

This is certify that the mini project based report entitled**"Design and Development of an Integrated Tourism Platform for Flights, Trains, Hotels, and Hidden Destinations"** is a bonafide work done and submitted by **P.RUHITHA – 2520080053 , M.ANANYA - 2520080069 CH. MADHULIKA – 2520090056** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of CSIT & AIDS, K L (Deemed to be University), during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr. K. Anusha**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

P. RUHITHA **2520080053**
M. ANANYA **2520080069**
CH. MADHULIKA **2520090056**

Abstract

Modern operating systems execute many processes concurrently, making process management an important responsibility of the operating system. This project presents a **Linux Process Monitoring and Control System** that provides a practical way to observe running processes and perform selected process-management operations through an interactive application.

The system obtains process information from the Linux /proc filesystem. It identifies readable process directories and collects important details such as Process ID (PID), process name, process state, parent process ID, user, command line, CPU usage, memory usage, and process runtime. The monitoring component also handles the dynamic nature of Linux processes, where a process may disappear while its information is being read.

The system provides three main control operations for a selected process. **SIGSTOP** is used to pause a process, **SIGCONT** is used to resume a stopped process, and **SIGTERM** is used to request process termination. The implementation validates process IDs and handles permission-related failures and unavailable processes appropriately.

The graphical interface presents process information in an organized manner and provides features such as process search, refresh operations, detailed process information, status messages, and controls for the supported process operations. CPU and memory usage are also monitored to help users understand how processes utilize system resources.

The project includes automated tests for the process manager, process controller, and graphical interface. Overall, the system connects operating-system theory with a working Linux implementation and provides a practical understanding of process identification, process states, resource monitoring, the /proc filesystem, Linux signals, permissions, error handling, and process-control mechanisms.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8-9
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		10-12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13-14
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		15-16
		4.1 Python Programming	
		4.2 Python Runtime Environment	
		4.3 Linux System Calls	
		4.4 Modular Application Design	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		17-20
		5.1 Overall System Architecture	
		5.2System Workflow	

		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**21-22**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core Python Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**23-28**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**29-30**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**31**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

An operating system manages processes that execute concurrently and provides mechanisms to observe and control them. Understanding these mechanisms is essential for studying process management and the interaction between applications and the Linux kernel. The Linux Process Monitoring and Control System is designed as a practical project for observing running processes and performing selected control operations. The system reads process information from the Linux /proc filesystem and presents it through a user interface.

1.2 Problem Statement

Managing multiple running processes requires users to identify processes, inspect their current state and resource usage, and perform control operations when necessary. Existing system utilities provide such functionality, but they can hide the underlying operating-system mechanisms from beginners. Therefore, the project aims to provide a simple practical application that demonstrates how Linux exposes process information and how processes can be controlled using signals and system-level interfaces.

1.3 Scope of the Project

The project covers discovery of numeric process directories under /proc, collection of process information, CPU and memory monitoring, searching and viewing process details, and control of a selected process. The supplied implementation provides pause, resume, and terminate operations and handles cases such as invalid PIDs, permission failures, and processes that disappear during monitoring.

1.4 Objectives of the Project

• Display information about currently running processes, such as Process ID, process name, and process state.

• Allow users to view basic and detailed information about a selected process using Linux process data.

• Monitor CPU and memory usage of processes.

• Provide pause, resume, and terminate operations for a selected process.

• Understand and demonstrate Linux process management, signals, system calls, and POSIX mechanisms through a practical implementation.

• Provide a searchable interface and validate the implementation through automated tests.

1.5 Importance of the Application

The application connects theoretical operating-system concepts with an observable implementation. It helps learners understand process IDs, process states, parent-child relationships, CPU and memory usage, the /proc filesystem, Linux signals, and system-level APIs in a practical manner.

It provides a simple way to observe running processes and understand how they can be monitored and controlled in a Linux environment. The application therefore helps students relate classroom concepts to actual operating-system operations.

1.6 Target Users / Audience

The primary audience is students and learners studying operating systems and Linux system programming. The application can be used to understand process monitoring, process states, resource usage, and basic process-control operations.

It can also be used as a practical demonstration tool in laboratory sessions and academic projects to explain basic Linux process monitoring and control concepts.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process Management

Process management is a major function of an operating system. It involves creating, scheduling, monitoring, and controlling processes during their execution.

Process Identification

Every running process in Linux is assigned a unique Process ID (PID). The PID is used to identify and perform operations on a particular process.

Process States

A process can exist in different states such as running, sleeping, stopped, or zombie. Monitoring these states helps in understanding the current condition of a process.

Process Control

Linux provides mechanisms to control processes using signals. In this project, SIGSTOP, SIGCONT, and SIGTERM are used to pause, resume, and request termination of a selected process.

Resource Monitoring

The operating system manages system resources such as CPU and memory. The project monitors CPU and memory usage to provide information about the resources consumed by individual processes.

Linux /proc Filesystem

The /proc filesystem provides process and system information in Linux. The project uses /proc to read details such as PID, process state, parent process ID, memory information, and command-line details.

2.2 Existing Methods / Algorithms

Process Monitoring Using /proc

Linux provides the /proc filesystem as an interface for accessing information about the running system and its processes. Each active process is represented by a directory identified by its numeric Process ID (PID). The application scans these process directories and reads the required information from the available files. Details such as process name, process state, parent process ID, user, command line, and other process-related information are collected and displayed to the user. This method provides a direct way of obtaining process information from the Linux operating system.

CPU Usage Monitoring

CPU usage monitoring helps in understanding how much processor time is being used by an individual process. The application collects process CPU-time information at different points in time and compares successive samples to determine the CPU usage. By using successive measurements rather than a single value, the system can provide an indication of the changing CPU activity of a process. This information is useful for observing the resource consumption of running processes.

Memory Usage Monitoring

Memory usage monitoring provides information about the amount of system memory occupied by a process. The application obtains resident memory information from the Linux process data and compares it with the total available system memory. This allows the memory usage of individual processes to be represented as a percentage. Monitoring memory usage helps users understand how different processes consume system resources while they are running.

Signal-Based Process Control

Linux signals provide a standard mechanism for controlling and communicating with processes. The project uses selected signals to perform basic process-control operations. SIGSTOP is used to pause a selected process, SIGCONT is used to resume a stopped process, and SIGTERM is used to request the termination of a process. These operations allow users to observe the effect of process-control mechanisms through the graphical application.

Process Information Parsing

The information obtained from the /proc filesystem is read and parsed to extract the required process details. The application processes information such as PID, process name, process state, parent PID, user, command line, CPU usage, memory usage, and runtime. The parsed information is then organized into a structured form for display in the application. Since processes can start or terminate at any time, the parsing method also needs to handle unavailable or changing process information appropriately.

2.3 Proposed Approach

The proposed system is designed to provide a simple and interactive way to monitor and control running processes in a Linux environment. The application uses the Linux /proc filesystem to obtain information about active processes. It identifies the available process directories and collects details such as Process ID (PID), process name, process state, parent process ID, user, CPU usage, memory usage, runtime, and command line. This information is organized and displayed through a graphical interface so that users can easily observe the current status of running processes.

The application also provides selected process-control operations using Linux signals. A user can select a specific process and use SIGSTOP to pause the process, SIGCONT to resume a stopped process, or SIGTERM to request its termination. The system validates the selected PID and handles situations such as invalid process IDs, permission-related failures, unavailable /proc information, and processes that disappear while their information is being collected.

The proposed approach separates the application into different components for process monitoring, process control, and the graphical interface. This modular structure makes the system easier to understand, test, and maintain. The graphical interface provides process search, refresh operations, detailed process information, status messages, and controls for the supported operations. Automated tests are also included for the major components to verify process information handling, control operations, validation, error handling, and interface behavior.

2.4 Advantages of the Proposed Approach

The proposed approach provides a simple and practical way to understand Linux process monitoring and control. By using the /proc filesystem, the system can obtain process-related information directly from the Linux operating system and display it in an organized manner. This helps users observe process states, CPU usage, memory usage, and other important process details through a single interface.

The graphical interface makes the system easier to use compared with performing process-monitoring operations only through command-line tools. Users can search for processes, refresh the displayed information, view process details, and perform supported control operations on a selected process. The use of SIGSTOP, SIGCONT, and SIGTERM also provides a practical understanding of Linux process-control mechanisms.

Another advantage is the modular design of the application. Process monitoring, process control, and the graphical interface are separated into different components, making the system easier to understand, test, and maintain. The application also includes validation and error handling for invalid PIDs, permission issues, unavailable process information, and processes that may disappear during monitoring.

Overall, the proposed approach provides an educational and practical environment for studying operating-system process management. It connects theoretical concepts with actual Linux operations while providing a controlled interface for observing and managing processes.

CHAPTER 3 – SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The Linux Process Monitoring and Control System does not require high-end or specialized hardware. It can be implemented and executed on a standard computer with sufficient processing power, memory, and storage to run the Linux operating system, the Python application, and the required supporting processes. A basic computer configuration is sufficient for monitoring processes, displaying process information, and performing the supported process-control operations smoothly.

Hardware Component	**Minimum Requirement**
Processor	Dual-Core Processor or above
RAM	4 GB or above
Storage	At least 10 GB of available space
Display	Standard monitor with suitable screen resolution
Input Devices	Keyboard and Mouse

3.2 Software Requirements

The software requirements for the Linux Process Monitoring and Control System include the operating system, programming language, libraries, and testing tools required to develop and run the application. The system requires a Linux environment with Python and the necessary libraries to support process monitoring, graphical interface development, and process-control operations. The required testing tools are also used to verify the functionality and reliability of the application.

Software Component	**Requirement**
Operating System	Linux
Programming Language	Python
GUI Framework	Tkinter / ttk
Process Information	Linux /proc filesystem
Process Control	Linux POSIX signals
Testing Framework	Python unittest
Development Environment	Python-supported IDE or text editor

3.3 Operating System Environment – Ubuntu / WSL

The Linux Process Monitoring and Control System is designed to operate in a Linux environment such as Ubuntu. Ubuntu provides the required operating-system facilities, including the /proc filesystem for accessing process information and Linux signals for controlling processes. The application uses these features to obtain details such as PID, process name, process state, parent process ID, CPU usage, and memory usage.

The project can also be executed using Windows Subsystem for Linux (WSL), which provides a Linux environment within Windows. WSL allows the required Linux commands, Python programs, filesystem features, and process-management operations to be used without requiring a separate physical Linux system. Therefore, Ubuntu or an appropriate Ubuntu-based WSL environment can be used for developing, testing, and running the application.

The /proc filesystem and POSIX/Linux signals used by the application are available in the Linux environment, making Ubuntu or an appropriate WSL-based Ubuntu setup suitable for the development and execution of the project.

3.4 Development Tools and Compilers

The project is developed using Python along with the required libraries and development tools. Python is used to implement the process monitoring, process control, and graphical interface components of the system. Tkinter and ttk are used to create the graphical interface, while Python's unittest framework is used to test the major components and verify their expected behavior.

The project can be developed using a suitable Python-supported IDE or text editor. Since the supplied implementation is written in Python, a separate C or C++ compiler is not required for building the application. The Python environment and Linux system together provide the necessary tools and facilities for developing, executing, testing, and maintaining the Linux Process Monitoring and Control System.

These development tools together support the implementation, execution, testing, and maintenance of the Linux Process Monitoring and Control System.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

4.1 Python Programming

Python is the main programming language used to develop the Linux Process Monitoring and Control System. It is used to implement the process monitoring, process control, and graphical interface components of the application. The language provides a simple and modular approach for working with process information and Linux system features.

In this project, Python is used to read and process information from the Linux /proc filesystem, perform the supported signal-based process-control operations, calculate resource usage, and display the collected information through the graphical interface. The use of Python also supports testing and maintenance of the different modules of the application.

4.2 Python Runtime Environment

Python provides the runtime environment required to execute the application. The Python interpreter runs the program modules responsible for process monitoring, process control, and the graphical interface. It also provides access to the libraries required for interacting with the Linux environment and performing the operations defined in the project.

Since the application is developed in Python, a GCC compiler is not required for compiling the project. The Python runtime environment is sufficient to execute, test, and maintain the supplied implementation.

4.3 Linux System Calls

Linux system-level interfaces are used by the application to interact with running processes and obtain process-related information. The project uses Linux facilities such as the /proc filesystem to read details about active processes and uses the operating system's signal mechanism to perform supported process-control operations.

These operations allow the application to communicate with the Linux operating system for monitoring and controlling selected processes. The system also handles conditions such as invalid PIDs, permission errors, and processes that are no longer available, ensuring that such errors do not terminate the complete application.

4.4 Modular Application Design

The application is organized into separate modules for process monitoring, process control, and the graphical user interface. The process manager is responsible for collecting and processing information from the Linux /proc filesystem, while the process controller handles the supported signal-based operations. The graphical interface provides the interaction between the user and these components.

This modular structure separates different responsibilities within the application and makes the system easier to understand, test, maintain, and modify. It also allows individual components to be tested independently without affecting the complete application.

4.5 Linux Commands and Utilities

Linux commands and utilities provide useful support for developing and verifying the process monitoring system. Standard Linux process-related utilities can be used to observe running processes and compare the information displayed by the application with the operating system. These utilities help in understanding process IDs, process states, resource usage, and process-control operations.

The Linux environment also provides the /proc filesystem and signal mechanisms required by the application. These facilities support the collection of process information and the execution of supported process-control operations.

4.6 Other Tools / Libraries Used

The project uses Tkinter and ttk libraries for developing the graphical user interface. These libraries provide the interface components required to display process information, search and refresh processes, show process details, and provide controls for supported operations.

Python's unittest and mocking facilities are used for testing the process manager, process controller, and graphical interface. The project is organized into separate modules, which makes the application easier to develop, test, and maintain.

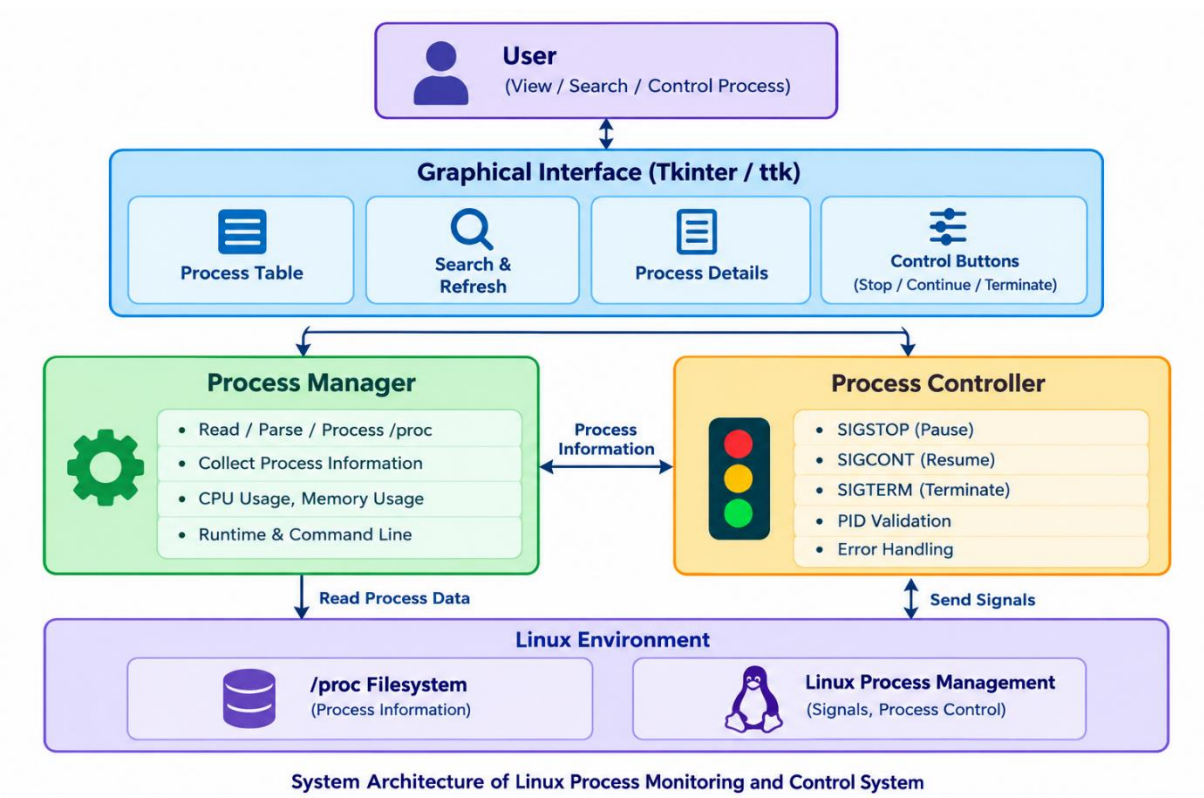
CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The Linux Process Monitoring and Control System follows a modular architecture consisting of the user interface, process monitoring component, process control component, and Linux operating-system facilities. The user interacts with the system through the graphical interface developed using Tkinter/ttk. The interface provides features such as viewing the process table, searching and refreshing processes, viewing process details, and performing supported process-control operations.

The Process Manager is responsible for obtaining and processing information about running processes from the Linux /proc filesystem. It collects details such as process ID, process information, CPU usage, memory usage, runtime, and command-line information. The Process Controller handles the control operations for a selected process using Linux signals such as SIGSTOP, SIGCONT, and SIGTERM. It also performs PID validation and handles errors that may occur during process-control operations.

The Linux environment provides the underlying facilities required by both components. The /proc filesystem supplies process information, while Linux process-management mechanisms handle the signals and control operations. The modular architecture separates monitoring, control, and user interaction, making the system easier to understand, test, and maintain.

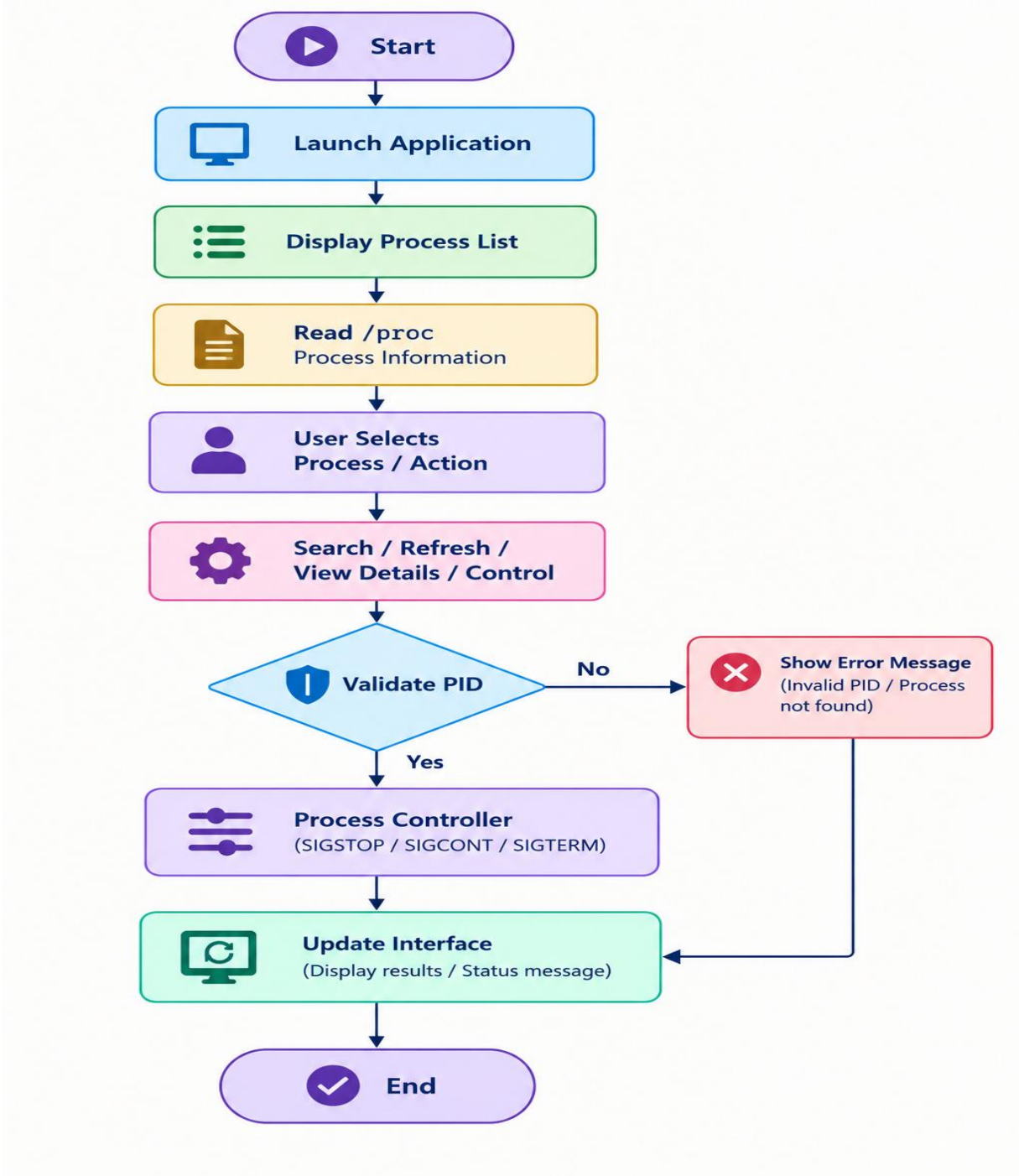


System Architecture of Linux Process Monitoring and Control System

5.2 System Workflow

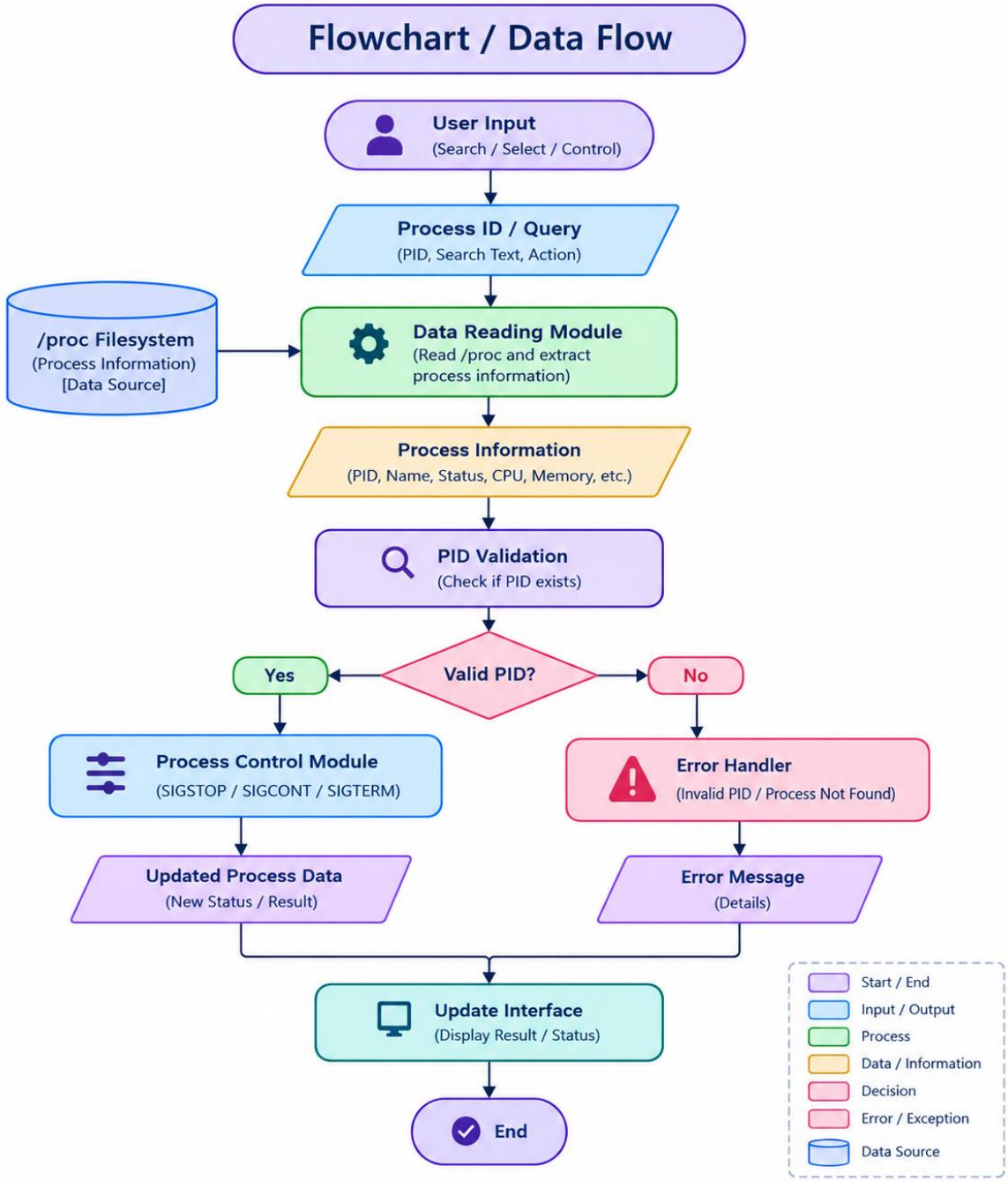
The system starts when the user launches the application and the graphical interface displays the available running processes. The Process Manager reads process information from the Linux /proc filesystem and displays details such as PID, process name, process state, CPU usage, memory usage, and runtime.

The user can search for a process, refresh the process list, or select a process to view its details. When a control operation is selected, the Process Controller validates the PID and sends the appropriate Linux signal, such as SIGSTOP, SIGCONT, or SIGTERM. The result of the operation is then reflected through the graphical interface with an appropriate status message.



5.3 Flowchart / Data Flow

The system starts by launching the application and displaying the list of currently running processes. It reads process information from the /proc filesystem and allows the user to search, refresh, view process details, or perform control actions. When a user selects a process, the system validates the Process ID (PID). If the PID is valid, the process controller performs actions such as SIGSTOP, SIGCONT, or SIGTERM. If the PID is invalid or the process is not found, an error message is displayed. Finally, the interface is updated with the process result or status, and the system ends the operation.



5.4 Module Description

1. Process Monitoring Module
Collects process information from /proc and displays PID, process name, status, CPU usage, and memory usage.

2. Process Search and Details Module
Allows users to search by process name or PID, view details, and refresh the process list.

3. PID Validation Module
Checks whether the selected PID is valid and available before a control operation.

4. Process Control Module
Handles SIGSTOP, SIGCONT, and SIGTERM operations for the selected process.

5. Error Handling Module
Handles invalid PIDs, unavailable processes, permission problems, and unsuccessful operations.

6. User Interface Module
Provides the Tkinter/ttk interface for displaying processes, searching, viewing details, and performing controls.

7. Status and Result Module
Updates the interface with the result or status of each operation.

5.5 Input and Output Design

Input Design

The system accepts input from the user through the application interface. The main inputs include:

- **Process ID (PID)** – identifies the process to be monitored or controlled.
- **Search Text –** used to find a specific process.
- **Process Selection** – allows the user to select a process from the displayed list.
- **Control Action** – the user can select actions such as Stop, Continue, or Terminate.
- **Refresh Request** – updates the process list with the latest process information.

Output Design

The system provides the following outputs:

- **Process List –** displays currently running processes and their details.
- **Process Information –** shows PID, process name, status, CPU usage, and memory information.
- **Control Result** – displays whether the selected process action was successfully performed.
- **Updated Process Status** – shows the latest status after a control operation.
- **Error Message** – informs the user when an invalid PID, unavailable process, or unsuccessful operation is detected.

The input and output design is kept simple and user-friendly so that process monitoring and control operations can be performed efficiently.

CHAPTER 6 – IMPLEMENTATION

6.1 Module-wise Implementation

The system is implemented using separate modules for process monitoring, process control, graphical interface, and testing. The Process Manager reads process information from the Linux /proc filesystem and collects details such as PID, process name, process state, parent PID, user, CPU usage, memory usage, runtime, and command line.

The Process Controller performs the supported process-control operations using Linux signals. The Graphical User Interface, developed using Tkinter/ttk, provides process viewing, searching, refreshing, detailed information, and control options. The modules work together to provide the complete process monitoring and control functionality.

6.2 Algorithm / Pseudocode

The implementation follows a simple sequence for monitoring and controlling processes:

1. Start the application.
2. Read available process directories from /proc.
3. Collect process information for each valid process.
4. Calculate CPU and memory usage.
5. Display the process information in the GUI.
6. Allow the user to search, refresh, or select a process.
7. Validate the selected PID.
8. Perform the requested operation using the appropriate signal.
9. Display the updated status or error message.
10. Continue monitoring until the application is closed.

6.3 Core Python Implementation

Python is used as the main programming language for implementing the system. The core implementation consists of the process manager, process controller, and graphical interface modules. Python provides the required functionality for reading /proc information, performing process-control operations, handling errors, and connecting the different components of the application.

6.4 Linux System Interfaces / APIs Used

The application uses Linux system facilities to monitor and control processes. The /proc filesystem is used to obtain process information, while Linux signal mechanisms are used for process control. The supported signals are SIGSTOP for stopping a process, SIGCONT for continuing a stopped process, and SIGTERM for requesting process termination.

before performing these operations, the selected PID is validated and possible conditions such as invalid PIDs, permission errors, or unavailable processes are handled appropriately.

6.5 Process / Thread Implementation

The project does not implement separate application-level process or thread management. Instead, it monitors the processes already running in the Linux operating system and performs selected control operations on them. The application interacts with these processes through the Linux process-management mechanisms.

6.6 IPC / Synchronization Implementation

The project does not use a separate IPC or synchronization mechanism. Communication with the operating system is performed through the Linux /proc filesystem and signal-based process-control interfaces. Therefore, mechanisms such as pipes, shared memory, message queues, or thread synchronization are not part of the supplied implementation.

6.7 Memory Management Implementation

The system monitors memory usage of running processes by obtaining resident memory information from /proc. The collected memory information is processed and represented as a percentage of the total system memory. This allows the user to observe the memory usage of individual processes through the graphical interface.

6.8 User Interface Implementation

The graphical interface is implemented using Python Tkinter/ttk. It provides the process table, search and refresh options, process details, control buttons, and status messages. The interface connects the user actions with the Process Manager and Process Controller, allowing process information to be viewed and supported control operations to be performed through the application.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

7.1 Setup and Execution in Ubuntu

The project was developed and tested in an Ubuntu/Linux environment because the application uses Linux-specific features such as the /proc filesystem and Linux process signals**.**

Before execution, Python 3 and Tkinter are required. On Ubuntu, Tkinter can be installed using:

```
sudo apt update
sudo apt install python3-tk
```

The Python source files can be checked for syntax errors and compiled using:

```
python3 -m compileall .
```

The project consists of the following major components:

```
OS_Project/
├── main.py
├── backend/
│   ├── __init__.py
│   ├── process_manager.py
│   └── process_controller.py
├── gui/
│   ├── __init__.py
│   └── main_window.py
└── tests/
    ├── test_process_manager.py
    ├── test_process_controller.py
    └── test_main_window.py
```

The project uses Python's standard library and Tkinter, so no additional Python packages are required.

7.2 Execution Commands

Step 1: Navigate to the project directory

cd /path/to/OS_Project

Step 2: Verify the project files

ls

Step 3: Verify Tkinter

python3 -c "import tkinter; print('Tkinter OK')"

Step 4: Execute the application

python3 main.py

Step 5: Run the automated test suite

python3 -m unittest discover -s tests -v

The project contains 44 automated tests, covering process management, process control, and GUI functionality.

Step 6: Run compilation checks

python3 -m compileall .

7.3 Sample Inputs and Outputs

The application is primarily GUI-based, so user interactions with the graphical interface serve as the main inputs.

1. Process Monitoring

python3 main.py

- Automatically detects running Linux processes
- Process table with PID, Process Name, User, State, CPU %, Memory %, Parent PID
- Statement that the values come from /proc

2. Process Search

- User enters python in the Search field
- System filters by PID, process name, or username
- Search is case-insensitive

3. View Process Details

- User selects a process
- Clicks View Details
- Details: PID, Process Name, State, User, Parent PID, CPU Usage, Memory Usage, Runtime, Full Command
- Then show your example details

4. Process Control

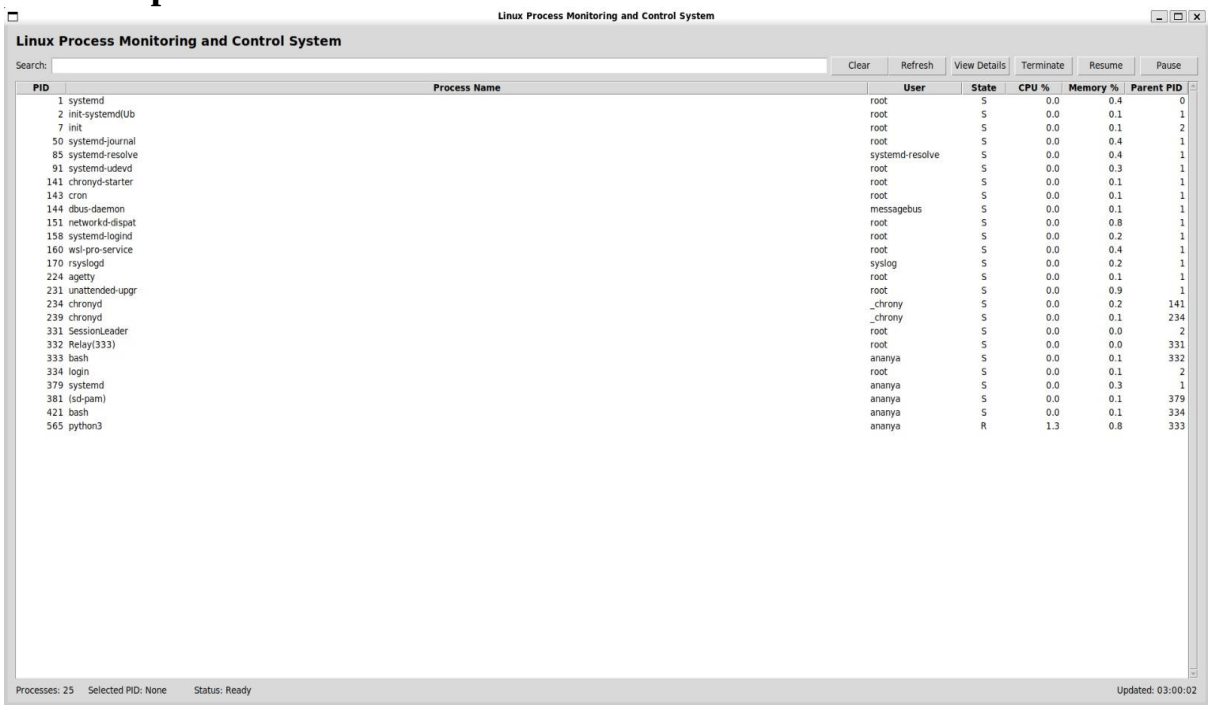
- sleep 300 &
- PID returned by Linux

Table:

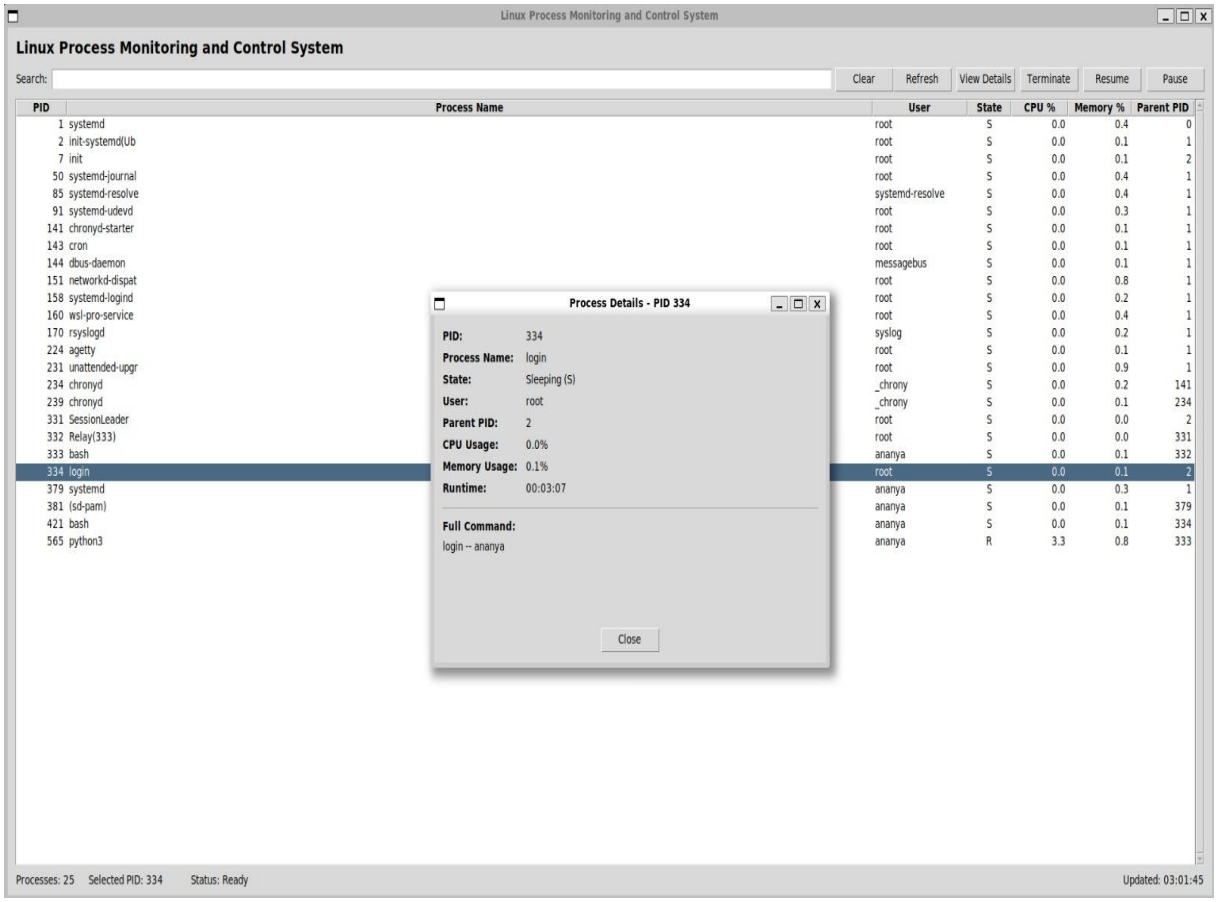
User Action	Linux Signal	Expected Result
Pause	SIGSTOP	Process is stopped
Resume	SIGCONT	Process continues
Terminate	SIGTERM	Process terminates

7.4 Screenshots of Execution

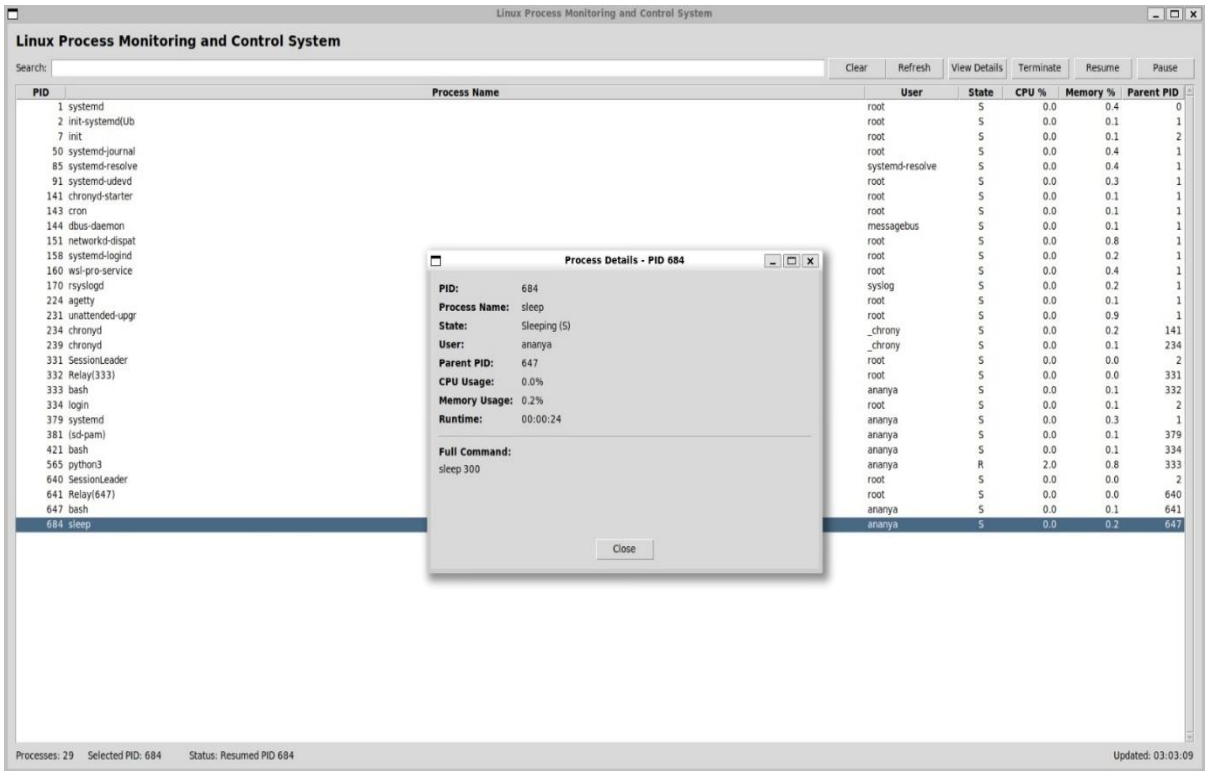
Search Operation



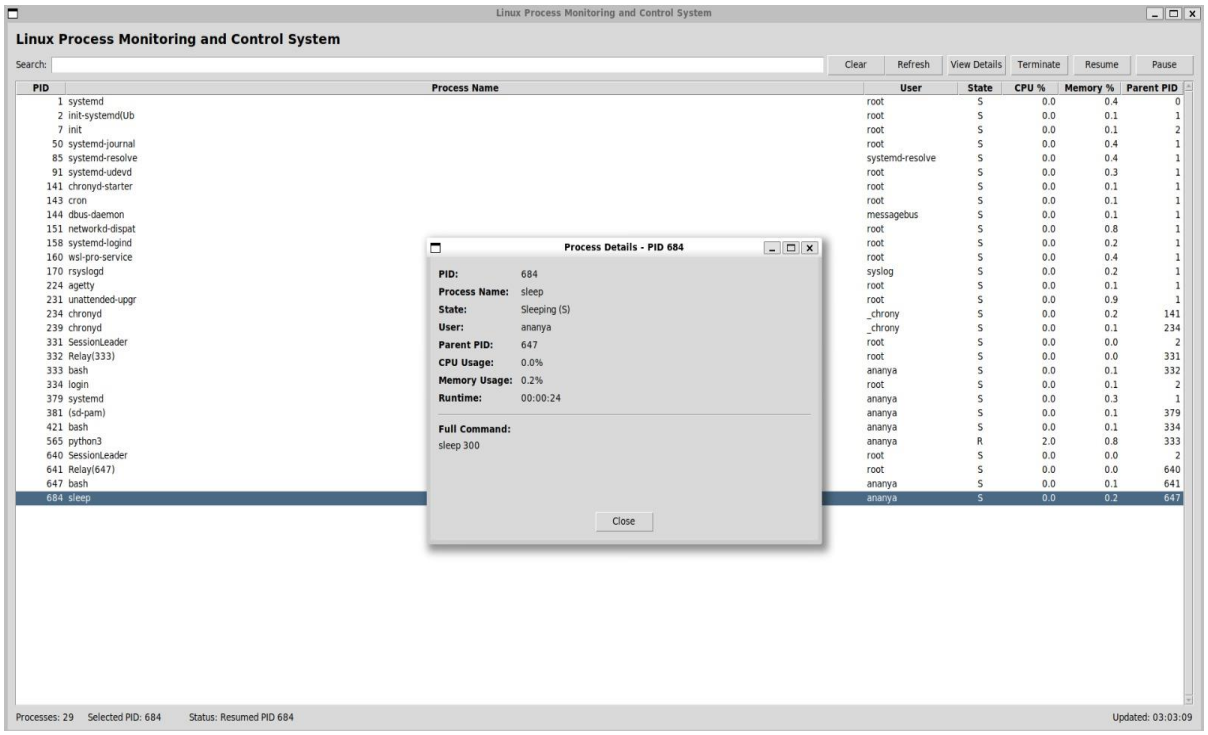
Process Details



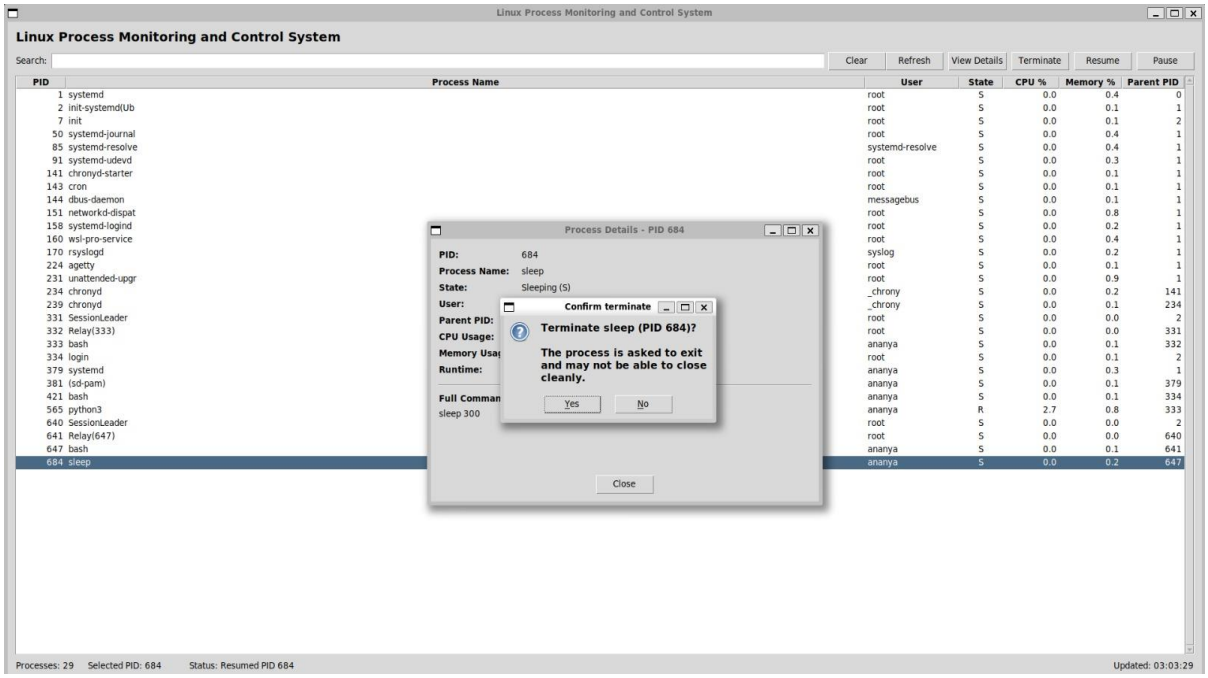
Pause



Resume



Terminate



7.5 Demonstration of OS Concepts

The Linux Process Monitoring and Control System demonstrates several important operating-system concepts through practical implementation in an Ubuntu/Linux environment.

1. Process Management

Process management is demonstrated by displaying currently running processes and their details. Each process is identified using a unique Process ID (PID). The application allows users to view process information and understand the current state of each process.

2. Process States

The application displays process states obtained from the Linux /proc filesystem. Processes may be running, sleeping, stopped, or in other operating-system states. This helps users understand how processes behave during execution.

3. Process Control Using Signals

The project demonstrates Linux signal-based process control through three signals:

- SIGSTOP: Stops a selected process.
- SIGCONT: Resumes a stopped process.
- SIGTERM: Requests termination of a process.

These operations demonstrate how the operating system provides mechanisms to control process execution.

7.5.4 CPU and Memory Monitoring

The application collects process-related information to monitor CPU and memory usage. CPU usage helps users understand processor activity, while memory usage indicates the amount of physical memory occupied by a process.

5. Linux /proc Filesystem

The /proc filesystem provides information about running processes and system resources. The application reads process information from this filesystem and displays the collected details through its graphical interface.

6. Error Handling and Process Validation

The application validates process IDs before performing control operations. It also handles invalid PIDs, permission errors, and processes that become unavailable during monitoring. This demonstrates the importance of error handling in system-level applications.

7. Demonstration Summary

The project connects operating-system theory with practical Linux operations. It demonstrates process identification, process states, resource monitoring, Linux signals, and process-control mechanisms through an interactive application.

CHAPTER 8 – CONCLUSION

8.1 Summary of the Project

The Linux Process Monitoring and Control System was developed to provide a simple and interactive way to monitor and control processes in a Linux environment. The application collects process information from the /proc filesystem and displays details such as Process ID, process name, process state, parent process ID, CPU usage, and memory usage.

The system provides a graphical interface developed using Python Tkinter/ttk. Users can search for processes, refresh the process list, view detailed process information, and perform supported process-control operations.

The application uses Linux signals such as SIGSTOP, SIGCONT, and SIGTERM to demonstrate process control. It also includes process validation and error handling to manage situations such as invalid process IDs, permission restrictions, and unavailable processes.

Overall, the project provides a practical understanding of operating-system process management and demonstrates how Python can interact with Linux system facilities.

8.2 Objectives Achieved

The main objectives of the project were addressed through the development of the process monitoring and control application.

1. Process Monitoring: The application collects and displays information about running processes in Linux.
2. Process Identification: Processes are identified using their unique Process IDs.
3. Resource Monitoring: CPU and memory usage information is collected and displayed.
4. Process Control: The application supports pause, resume, and termination requests using Linux signals.
5. Graphical Interface: A Tkinter/ttk interface provides process searching, refreshing, and detailed information.
6. Error Handling: The system handles invalid PIDs, permission issues, and unavailable process information.
7. Testing: Automated tests are provided for the process manager, process controller, and graphical interface.

8.3 Results Achieved

The project provides an interactive application for observing running Linux processes and performing selected process-control operations.

The major results achieved are:

- Process information is collected from the Linux /proc filesystem.
- Process details are presented in an organized graphical interface.
- Users can search for processes and view their detailed information.
- CPU and memory usage information is displayed for process monitoring.
- Supported process-control operations are performed using Linux signals.
- Error-handling mechanisms help manage invalid or unavailable processes.
- Automated tests support the verification of the application's major components.

The project demonstrates how operating-system concepts can be implemented in a practical application. The final results can be verified through the application interface, execution screenshots, and test results.

8.4 Skills and Learning Outcomes

The development of this project helped strengthen knowledge of operating systems, Linux commands, Python programming, and graphical application development.

The main learning outcomes include:

- Operating-System Concepts: Understanding process management, process states, process identification, and resource monitoring.
- Linux System Programming: Learning how Linux provides process information through the /proc filesystem and supports process control through signals.
- Python Programming: Developing modular Python code for process monitoring, control operations, and error handling.
- GUI Development: Understanding how to create interactive application interfaces using Tkinter and ttk.
- Testing and Debugging: Learning how to test individual modules and handle errors during execution.
- Modular Design: Understanding the benefits of separating process monitoring, process control, and user-interface functionality.
- Practical Problem-Solving: Applying theoretical operating-system concepts to a working Linux application.

CHAPTER 9 – REFERENCES

9.1 Books

1. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, Wiley.
2. William Stallings, Operating Systems: Internals and Design Principles, Pearson.
3. Maurice J. Bach, The Design of the UNIX Operating System, Prentice Hall.

9.2 Research Papers

1. Ritchie, D. M., and Thompson, K. (1974). "The UNIX Time-Sharing System." Communications of the ACM, 17(7), 365–375.
2. Saltzer, J. H., and Kaashoek, M. F., Principles of Computer System Design: An Introduction. MIT Press.

These references provide background on UNIX systems, operating-system design, and process management. They are general background references, not claims that the project implements every technique discussed in them.

9.3 Linux / GNU Documentation

1. Linux man-pages project — proc(5), documentation for the /proc filesystem.
2. Linux man-pages project — kill(2), documentation for sending signals to processes.
3. Linux man-pages project — signal(7), overview of Linux signals.
4. Python documentation — tkinter, documentation for Python's standard GUI toolkit.
5. Python documentation — unittest, documentation for Python's unit-testing framework.

9.4 Online Resources

1. Python Official Documentation: https://docs.python.org/3/
2. Linux man-pages: https://man7.org/linux/man-pages/
3. Ubuntu Official Documentation: https://help.ubuntu.com/
4. GNU Project: https://www.gnu.org/
5. Python Tkinter Documentation: https://docs.python.org/3/library/tkinter.html